

Privacy and Network Systems: In-Kernel Packet Filtering, TLS Traffic Fingerprinting and Tor Path Selection

Vansil Rakeshkumar Chauhan (24M0847)

CS790: Privacy Enhancing Technologies
Instructor: Prof. Devashish Gosain
Spring Semester, January–April 2025

1 Objective

This report covers the three assignments of the course, which approach network privacy at three different layers of the same stack.

Assignment 1 works *below* the firewall, building a filtering policy with FreeBSD’s `pf` and then reimplementing the same class of filtering as a kernel module hooked directly into the IPv4 receive path. Assignment 2 works *above* the transport, identifying Tor traffic from the shape of its TLS handshake despite Tor’s deliberate efforts to resemble ordinary HTTPS. Assignment 3 works *inside* Tor itself, taking manual control of circuit construction and reimplementing the relay-selection algorithm from the live network consensus.

Although the three were set separately, working through them in sequence produced a common theme that I return to in the discussion: in each case the substantive work was not building the mechanism but establishing that the mechanism did what it appeared to do. Each assignment is also treated with its own limitations section below, because the honest boundary of what was built is as much a part of the result as the working parts.

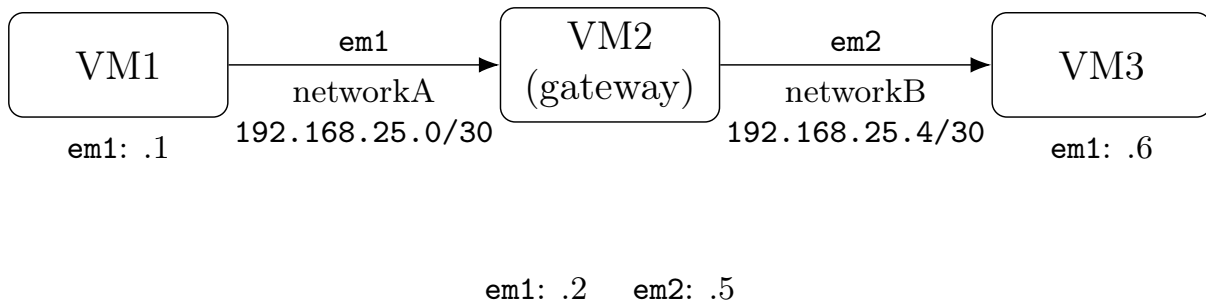
2 Assignment 1 — packet filtering on FreeBSD

The assignment had two tasks addressing packet filtering at different layers of one system. Task 1 builds a routed three-VM topology and enforces a policy on the gateway using `pf`, FreeBSD’s in-kernel packet filter. Task 2 implements the same class of filtering *below* `pf`, as a loadable kernel module hooked into the IPv4 receive path through the `pfil(9)` API.

The pairing is the point of the assignment. Task 1 uses the production tool; Task 2 builds the mechanism that the production tool is itself a consumer of, which makes it possible to measure one against the other.

2.1 Topology and routing

I built three VMs in a linear arrangement with the gateway in the middle, across two subnets: VM1 on 192.168.25.0/30, then VM2 as gateway, then VM3 on 192.168.25.4/30.



I chose /30 subnets deliberately. With a 30-bit prefix each segment has exactly two usable addresses, which is precisely a point-to-point link and nothing more. There is no room for a stray host, so the routing path is unambiguous by construction rather than by convention.

Each VM’s third-party interface, `em0`, carries a DHCP address on the host’s bridged adapter purely for SSH access from the host — it plays no part in the routed topology and every filtering decision below concerns only the `em1/em2` path. On VM2 I enabled forwarding with `gateway_enable` set to "YES" in `/etc/rc.conf` and set static routes on VM1 and VM3 pointing at VM2 for the far subnet (`static_routes` naming the peer network, `route_networkA/route_networkB` giving the next hop). I then confirmed with `tracert` in both directions that VM1–VM3 traffic genuinely transits VM2. That check matters more than it looks: without it, a permissive default route could produce working connectivity that bypasses the gateway entirely, and every subsequent firewall test would be measuring nothing. An Apache server on the protected VM, serving on port 80 from the default document root, provides the HTTP service the policy is written around.

2.2 Firewall policy and verification

`pf` is enabled with `pf_enable="YES"` and `pflog_enable="YES"` in `rc.conf`, and reloaded with `pfctl -nf /etc/pf.conf` (a dry-run syntax check) followed by `pfctl -f /etc/pf.conf` and `service pf restart`. The final `pf.conf` follows **default-deny**: block all first, then an explicit `pass` rule for TCP port 80 only:

```

block all
# allow http port 80 for internal network
pass in proto tcp from any to any port 80

```

Default-deny is the correct structure because the two orderings fail differently. Under default-permit a forgotten rule silently allows traffic; under default-deny a forgotten rule breaks a service, which is noisy and gets fixed. The failure mode is visible instead of invisible, and that asymmetry is worth more than the convenience of the alternative.

Getting to that final ruleset took one more iteration than expected. An intermediate version additionally carried `pass in proto icmp from any to any`, on the assumption that leaving ping open was harmless. Loading it showed SSH correctly blocked but ping still succeeding, which is exactly what a rule that explicitly passes ICMP should do — the firewall was behaving correctly, my definition of “blocked” was just looser than the rule I had written. Removing that line is the “ICMP tightening” step in the table below.

I tested each rule for both the traffic it should permit and the traffic it should block. Confirming that blocked traffic fails proves the rule is active; confirming that permitted traffic still succeeds proves the rule is not simply blocking everything. A test that checks only one side cannot distinguish a working firewall from a broken network.

Test	Before rules	After rules
HTTP (curl) to the web server	Succeeds	Succeeds
SSH to the protected host	Succeeds	Hangs / times out
ICMP (ping)	Succeeds	Blocked after ICMP tightening

2.3 An in-kernel filter with pfil(9)

`pfil(9)` is the kernel's packet filter *hook framework* — the set of interception points in the network stack where filtering code can attach. `pf`, `ipfw` and `ipfilter` are all consumers of it. Writing directly against `pfil(9)` means implementing a filter at the same layer as those tools rather than configuring one of them.

I create a hook with `pfil_add_hook()` typed `PFIL_TYPE_IP4` and `PFIL_IN`, then attach it to the kernel's default IPv4 inbound chain with `pfil_link()`. This is the same registration pattern that in-tree consumers use, which I cross-checked against `ip_fw_pfil.c` in the FreeBSD kernel source rather than reconstructing from documentation.

The filter function calls `m_pullup` to guarantee a contiguous IP header, checks `ip_p == IPPROTO_ICMP` and passes immediately otherwise, pulls up again for the ICMP header, checks whether `icmp_type` equals `ICMP_ECHO` so that echo *requests* specifically are caught while replies and other ICMP types pass untouched, and returns `PFIL_DROPPED` only in that case:

```

if (m->m_len < (int)sizeof(struct ip)) {
    m = m_pullup(m, sizeof(struct ip));
    if (m == NULL) { *m0 = NULL; return (PFIL_DROPPED); }
    *m0 = m;
}
ip = mtod(m, struct ip *);
if (ip->ip_p != IPPROTO_ICMP)
    return (PFIL_PASS);

ip_hlen = ip->ip_hl << 2;
if (m->m_len < ip_hlen + (int)sizeof(struct icmp)) {
    m = m_pullup(m, ip_hlen + sizeof(struct icmp));
    if (m == NULL) { *m0 = NULL; return (PFIL_DROPPED); }
    *m0 = m; ip = mtod(m, struct ip *);
}
icp = (struct icmp *)((caddr_t)ip + ip_hlen);
if (icp->icmp_type != ICMP_ECHO)
    return (PFIL_PASS);

icmp_dropped_count++;
printf("block_icmp: dropped ICMP Echo Request #%lu, size %u bytes\n",
    icmp_dropped_count, ntohs(ip->ip_len));
return (PFIL_DROPPED);

```

The repeated `m_pullup` calls are not redundant. Network data arrives as `mbuf` chains that may be fragmented across buffers at arbitrary boundaries, so a header is not guaranteed to be contiguous in memory. Casting a struct pointer over a chain boundary without pulling up first reads adjacent garbage, and each pullup guarantees contiguity only for the length requested — hence a second call once the ICMP header is known to be needed.

Each drop increments a counter and prints the count and packet size to the kernel console. I read the size as `ntohs(ip->ip_len)`, which is necessary because `ip_len` is still in network byte order at this point in the pipeline: `ip_input()` does byte-swap it, but into a local variable rather than back into the `mbuf`, and that swap happens *after* `pfil` hooks run. I confirmed this

by reading `ip_input.c` rather than assuming either way.

A version difference worth recording

The assignment lists FreeBSD 13.4 as preferred, but 13.4 images are no longer offered for download, so I used FreeBSD 15.1 and checked the `pfil(9)` API across the two versions before committing. FreeBSD split the old single `pa_func` hook callback into two typed variants, `pa_mbuf_chk` and `pa_mem_chk`. My filter function's signature matches `pfil_mbuf_chk_t` exactly, so the filter logic is unchanged and the adaptation was a one-line rename. Everything else used — `pfil_link_args`, `PFIL_TYPE_IP4`, `pfil_return_t`, and the `VNET_DECLARE` pattern — is identical between versions.

2.4 Load test

Building a kernel module requires kernel source, which the base install does not ship, so I fetched and extracted the matching 15.1-RELEASE `src.txz` first. The module then compiled cleanly with no warnings, producing `block_icmp.ko`, a 12,256-byte ELF 64-bit LSB relocatable object.

Step	Result
Baseline ping VM1 → VM3	0 % loss
<code>kldload ./block_icmp.ko</code> on VM2	100 % packet loss
<code>dmesg</code> on VM2, per echo request	dropped ICMP Echo Request # <i>n</i> , size 84 bytes (<i>n</i> = 1, 2, 3, ...)
HTTP fetch VM1 → VM3, module loaded	Succeeds
<code>kldunload block_icmp</code>	dropped 3 ICMP Echo Request packet(s) total; ping resumes at 0 % loss

The counter increments once per intercepted echo request rather than once per module load, which is what makes the unload-time total a meaningful checksum against the console log rather than a fixed constant. The HTTP check is the important row: it demonstrates a targeted ICMP filter rather than a general outage, holding Task 2 to the same verify-both-directions standard as Task 1.

2.5 Performance: the kernel module against pf

The module hooks *every* inbound IPv4 packet, not only ICMP — the type check happens inside the filter function, after an `m_pullup`, so a per-packet cost is paid even for traffic that is ultimately passed through unchanged. I wanted to know whether that cost is visible against `pf` doing the same job.

I measured TCP throughput with `iperf3` from VM1 to VM3 through the gateway, in 8-second runs with five repetitions per condition: no filtering, the module loaded, and an equivalent-scope `pf` rule (`pass all; block in quick proto icmp all`). I used that minimal ruleset rather than Task 1's default-deny policy deliberately, to isolate the cost of the ICMP-blocking mechanism itself rather than measuring a broader firewall policy.

The kernel module shows no measurable throughput cost — its mean sits inside the baseline's own run-to-run spread. `pf` costs a consistent 5–6 %, and the separation is clean: `pf`'s five runs span 237–248 and the module's span 252–262, with no overlap at all, which is meaningful despite the small sample.

Condition	Runs (Mbit/s, receiver)	Mean	Median
No filtering	264, 259, 252, 251, 236	252.4	252
<code>block_icmp.ko</code>	262, 259, 255, 252, 253	256.2	255
<code>pf</code> (equivalent scope)	239, 248, 243, 238, 237	241.0	239

The likely cause is state tracking. `pf`'s `pass` rules imply `keep state` by default, so every TCP segment performs a state-table hash insert or lookup. My hook does a direct `ip_p` and `icmp_type` comparison per packet with no state at all, because a stateless single-purpose check does not need any.

I want to be careful about what this does and does not show. It is not “custom code beats a real firewall”. `pf` provides a full stateful rule-matching engine — arbitrary protocol, port and flag matching, connection tracking, NAT — that a single-purpose hook does not replicate. The claim is narrower: purpose-built kernel code carrying no unnecessary generality can measurably outperform a general-purpose engine doing nominally equivalent work, and the price of that performance is everything the general engine would have provided.

2.6 Limitations

- **Small sample, single environment.** Five `iperf3` runs per condition, on virtualized hardware. The `pf`/module separation is clean, but a bare-metal repetition with more runs would strengthen it.
- **Throughput only.** No latency or packets-per-second measurement. Per-packet overhead should be far more visible in a small-packet PPS test than in a bulk TCP throughput test, which is dominated by large segments amortizing the per-packet cost.
- **No configurability.** The module is hardcoded to inbound IPv4 ICMP echo requests, with no runtime control interface — no `sysctl` to enable or disable it, no configurable set of types to block.
- **IPv4 only.** There is no `PFIL_TYPE_IP6` hook registered, so ICMPv6 echo requests pass untouched.
- **No concurrency handling on the counter.** `icmp_dropped_count` is a plain, non-atomic increment, which could undercount drops under genuine parallel packet reception across multiple queues.

3 Assignment 2 — Evil Eye: detecting Tor by TLS fingerprinting

The task was to build a Wireshark plugin that identifies Tor traffic in a capture and labels it. What makes this interesting is that Tor is deliberately designed to resist exactly this: it runs over TLS on arbitrary ports, so to casual inspection it looks like ordinary HTTPS. The problem is therefore not “find the Tor packets” but “find a signal that survives Tor’s efforts to blend in”.

3.1 Why the obvious approaches fail

Port-based detection. Tor commonly uses 9001 and 9030, but nothing requires it. Relays can listen anywhere and bridges routinely use 443 specifically to appear as normal HTTPS. A port filter catches only operators who did not change a default — the population least interested in evading detection.

IP-based detection against a relay list. The public list is available, but bridges exist *because* they are unpublished, and the list changes continuously, so any local copy is stale from the moment it is taken. This fails hardest against the users most motivated to evade it.

Both share the same defect: they identify *where* traffic goes rather than *what it is*, so both degrade as infrastructure changes underneath them. A signal derived from the traffic’s own structure does not.

3.2 Design: fingerprinting the handshake

When any TLS client opens a connection it sends a ClientHello advertising the cipher suites it supports, *in a specific order*. That list is determined by the client’s TLS implementation and configuration, not by the user and not by the destination. A browser, `curl`, a Python `requests` client and Tor each produce a characteristic signature, and Tor’s is consistent.

This is the principle behind JA3 fingerprinting: taking the ClientHello’s TLS version, cipher-suite list, extension list, elliptic-curve list and elliptic-curve point-format list, concatenating them in the order they appear on the wire, and hashing the result. JA3 uses all five fields; the plugin here narrows that to the single field that carries the most identifying signal for this specific target and is cheapest to check losslessly in Lua — the ordered cipher-suite list — rather than computing the full hash. The property that makes either version work is that the signal is intrinsic to the software rather than to its network location: it appears in the first packet of the connection, before any application data exists, and it cannot be changed without modifying the TLS implementation. It is also worth noting that Tor’s 18-entry list mixes three TLS 1.3 suites (0x1301–0x1303) with fifteen TLS 1.2 suites — the fingerprint captures the specific TLS library and build Tor ships with, not just a protocol-version preference, which is exactly why its order is distinctive rather than incidental.

3.3 Implementation

The plugin registers through `register_postdissector`, so it runs *after* Wireshark’s own TLS dissector has parsed the handshake. This is the decision I would defend most strongly in the plugin. It consumes Wireshark’s dissection output rather than reimplementing TLS parsing — which would have been substantial work, would need maintaining against protocol changes, and worst of the three, would risk disagreeing with the parser whose output the user is looking at in the same window.

Detection requires four independent conditions simultaneously: a TLS handshake record (`content_type` of 22); a ClientHello specifically (`handshake_type` of 1); a cipher-suites length of 36 bytes; and an **exact ordered match** against an 18-entry list of Tor’s cipher suites:

```
function is_tor_traffic(pinfo)
    if is_ip_sus(pinfo.src) then return true end
    if is_ip_sus(pinfo.dst) then return true end

    local content_type = tls_content_type() and tls_content_type().value
    local handshake_type = tls_handshake_type() and tls_handshake_type().value
    local cipher_suite_len = tls_cipher_suite_len() and tls_cipher_suite_len().value
    local cipher_suite_list = {tls_cipher_suite_list()}

    if content_type == 22 and handshake_type == 1
        and cipher_suite_len == 36
        and ciphers_match(tor_cipher_suites, cipher_suite_list) then
        tor_entry_ip[tostring(pinfo.dst)] = true
        return true
    end
end
```

```
end
return false
end
```

The fourth is what makes this a fingerprint rather than a heuristic: *order matters, not merely set membership*. Two implementations supporting the same suites will typically advertise them in different priority orders, so ordering carries substantially more identifying information than the set does. Requiring all four in conjunction is deliberate false-positive control — each signal alone is weak, since many clients advertise 36 bytes of cipher suites and every HTTPS connection begins with a ClientHello.

On a match I record the destination IP and flag every subsequent packet to or from that address without re-examining the handshake. This is necessary rather than an optimisation, for the same reason the fingerprint works: the signal appears *only* in the ClientHello, and every subsequent packet is encrypted application data with no distinguishing features. Without tracking, the plugin would label exactly one packet per connection and leave the session unmarked. The tracked address is also a Tor entry node — a guard or bridge — which is itself the useful intelligence.

Matches are annotated through registered `ProtoFields` rather than plain text, so Wireshark’s column and colouring rules can key off them and they are filterable like native fields.

3.4 Wireshark integration

Registering the fields is only half the deployment story — the plugin has to be surfaced to an analyst working in the ordinary Wireshark interface, not just to a filter expression. I added a custom column titled “Tor?” bound to `evil_eye_plugin.message`, so every packet list shows **Tor!** in-line without opening the protocol tree, and a colouring rule keyed on `evil_eye_plugin.is_tor == true` so matched packets are visually distinct in the packet list itself. Both update automatically as the plugin runs, because they read the same `ProtoFields` the post-dissector sets rather than a separate summary the plugin would have to maintain in sync.

3.5 Results

The plugin was validated against real Tor Browser sessions rather than synthetic captures, with a mixed-traffic capture as the demonstration. That distinction matters: a synthetic capture would only confirm that the plugin matches the list it was given, which is a tautology. Real traffic confirms that the list corresponds to what Tor actually emits, which is the claim the plugin rests on.

3.6 Dual use and the countermeasure

Fingerprinting inverts the usual privacy assumption. Encryption protects the *contents* of a connection, but the metadata of negotiating that encryption is transmitted in the clear and is characteristic of the software that produced it. A user can encrypt everything they send and still be identified by *how* they set up the encryption.

This is dual-use, and that is the point of the exercise rather than an incidental side effect. The same technique that lets a defender detect Tor on a corporate network lets a censor identify and block Tor users in a repressive jurisdiction. That tension is real in privacy-enhancing technology research generally: understanding an attack is a prerequisite for defending against it, and Tor’s own pluggable-transport work exists specifically because attacks of this class are effective in practice.

The countermeasure follows directly from the mechanism. Since the fingerprint *is* the handshake, the defence is to change the handshake — which is what pluggable transports do: **obfs4** wraps the connection to look like uniformly random bytes rather than any recognisable protocol, **meek** tunnels Tor traffic inside an HTTPS connection to a large cloud provider so that blocking it means blocking the provider, and **Snowflake** routes through ephemeral browser-based proxies. A detector built on a fixed cipher-suite fingerprint is defeated by a client that no longer produces that fingerprint, which is exactly the property these transports are designed to break.

3.7 Limitations

- **No measured false-positive rate.** Minimizing false positives was a design goal and a testing category, but no rate was quantified against a large corpus of ordinary HTTPS traffic. Running the plugin over a substantial general-browsing capture and counting spurious matches is the most valuable outstanding measurement.
- **The fingerprint is a point-in-time snapshot.** The hardcoded 18-entry cipher list reflects Tor’s TLS stack as of early 2025. Tor updates its cryptographic configuration over time, so the plugin requires maintenance to remain accurate — and would report clean on a newer Tor client rather than failing visibly.
- **Defeated by pluggable transports by design,** as above: obfs4, meek and Snowflake exist precisely to defeat this class of detection, and the plugin makes no attempt to detect them.
- **Exact matching is brittle in both directions.** Requiring an exact ordered match minimizes false positives but means any variation — a different Tor version, a non-default configuration, a different TLS library build — is missed entirely. A similarity score over the cipher list would trade some precision for robustness.
- **IP-based tracking has a lifetime problem.** `tor_entry_ip` is retained without expiry, so an address reassigned to a different host later in a long capture would continue to be labelled.

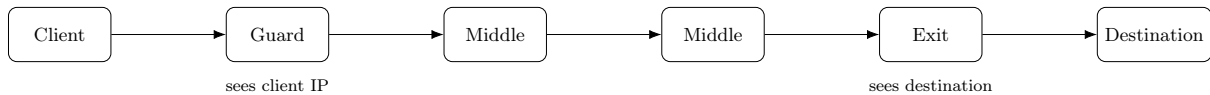
4 Assignment 3 — Tor circuits and relay selection

Tor routes traffic through a three-hop circuit chosen automatically, and for most users the selection algorithm is invisible. This assignment required opening that box in two directions: taking manual control of circuit construction, and reimplementing the selection algorithm itself.

The second is the substantive half. Calling a library to build a circuit demonstrates familiarity with an API; reconstructing bandwidth-weighted path selection from the consensus demonstrates understanding of why Tor performs acceptably at all given a relay population of wildly unequal capacity.

4.1 Background: how Tor picks a path

A circuit has three positions with different trust properties. The **guard** sees the client’s real IP, so clients stick with a small long-lived guard set — repeatedly rotating entry points would eventually route a client through a hostile one. The **middle** sees neither end. The **exit** contacts the destination and sees plaintext for non-TLS traffic.



No single relay in the chain sees both the client’s identity and the destination, which is the property the circuit exists to guarantee; the diagram above corresponds to the four-hop circuit built in Task 1, where a second middle relay is inserted between the three canonical positions.

The consensus is the document that makes coordinated path selection possible at all: a signed, hourly-regenerated list on which the directory authorities agree, recording every relay’s identity, status flags and measured bandwidth. Every client selects paths from the same consensus, which matters — if clients had differing views of the network, path choices would themselves be fingerprintable.

Selection is **bandwidth-weighted**, **not uniform**, and this is the detail that matters most. Tor’s relays range from home connections to multi-gigabit servers, and uniform selection would saturate small relays while leaving most capacity idle. Weighting by advertised bandwidth matches load to capacity instead.

That single decision is what makes Tor usable rather than merely functional, and it carries a measurable anonymity cost: high-bandwidth relays appear in disproportionately many circuits, so an adversary operating one sees a disproportionate share of traffic. Performance and anonymity are traded against each other directly, inside the algorithm.

4.2 Manual circuit construction

The script connects to Tor’s control port through `stem` and retrieves the live relay list. I choose relays subject to Tor’s own role constraints rather than at random — first hop **Guard**, last hop **Exit**, all hops **Fast**, **Running**, **Stable** and **Valid** — and build a four-hop path directly through the control port, retrying up to 15 times since relays in the consensus may be transiently unreachable and extension can fail at any hop.

Proving the circuit carries traffic took more effort than building it. By default Tor attaches each new stream to a circuit of its own choosing, so building a custom circuit and then issuing a request proves nothing: the request would most likely travel over a different, automatically-selected circuit while mine sat idle. The demonstration would look successful and mean nothing.

I therefore set `__LeaveStreamsUnattached` so Tor stops auto-attaching streams, register a listener for new-stream events, and attach each stream to my circuit explicitly:

```

def tor_custom_get_request(controller, url, circ):
    controller.set_conf('__LeaveStreamsUnattached', '1')
    stream_attached = threading.Event()

    def attach_stream(stream):
        if stream.status == 'NEW':
            controller.attach_stream(stream.id, circ)
            stream_attached.set()

    controller.add_event_listener(attach_stream, EventType.STREAM)
    try:
        response_thread = threading.Thread(
            target=lambda: requests.get(url, proxies=proxies))
        response_thread.start()
        stream_attached.wait(timeout=10)
        response_thread.join()

```

```
finally:
    controller.remove_event_listener(attach_stream)
    controller.reset_conf('__LeaveStreamsUnattached')
```

With that in place an HTTP GET through the SOCKS proxy is demonstrably carried by the specific four-hop path, which I confirmed live with `nyx`.

Task 1's relay list comes from `controller.get_network_statuses()` — Tor's own internal view of the consensus, already parsed and cached by the locally running daemon. This is a deliberate difference from Task 2, described next, which throws that convenience away and re-derives the same information from the raw published document. The two scripts therefore demonstrate two different levels of trust in Tor's own machinery: one uses it, the other independently reconstructs it.

4.3 Relay-selection simulator

The simulator scrapes Tor's Collector service for the most recent consensus and parses its line-oriented format from scratch — `r` lines for relay identity, `s` for status flags, `w` for bandwidth weight — converting fingerprints from base64 to the hex form the control protocol expects. Writing the parser by hand rather than using an existing library was deliberate: the format *is* the network's shared state, and parsing it myself is what makes the selection logic meaningful rather than opaque.

The function `weighted_choice` implements selection probability proportional to bandwidth through a cumulative-weight draw: bandwidths accumulate into a running total, a uniform value is drawn over that total, and the relay whose interval contains it is selected.

```
def weighted_choice(relays):
    total_bw = sum(r.bandwidth for r in relays if r.bandwidth > 0)
    if total_bw == 0:
        return random.choice(relays)
    r = random.uniform(0, total_bw)
    upto = 0
    for relay in relays:
        if relay.bandwidth <= 0:
            continue
        upto += relay.bandwidth
        if upto >= r:
            return relay
    return random.choice(relays)
```

Three role-specific selectors filter to the flags Tor's specification requires for each position *before* applying the weighting, and exclude relays already used — repeating one would collapse the trust separation that having three distinct positions exists to provide. `create_n_hop_circuit` generalises this to arbitrary path length, making the four-hop requirement a special case.

4.4 Path length and the anonymity property

A natural assumption is that a longer circuit is stronger. It is not, in the threat model Tor targets. Tor's anonymity rests on no single relay knowing both the client and the destination, and three hops already guarantee that: an additional middle hop adds latency and consumes network capacity without changing the property that makes the circuit anonymous. Against a global passive adversary capable of end-to-end traffic correlation, extra hops do not help either — correlation attacks observe the two ends of the circuit, and the path length between them

is irrelevant to that observation. Building a four-hop circuit here is a useful exercise in taking manual control of path construction, not a security improvement over Tor’s own default.

4.5 Results

Deliverable	Outcome
Custom 4-hop circuit construction	Working; validated live via <code>nyx</code>
Traffic proven to traverse the chosen circuit	Working, via manual stream attachment
Live consensus retrieval and parsing	Working against the real Collector service
Bandwidth-weighted role-constrained selection	Working, rebuilt from the format up
Arbitrary n -hop circuits	Working

4.6 Limitations

- **No bounded guard set.** Real clients persist a small guard set across circuits precisely to limit exposure to a hostile entry relay; this simulator selects a fresh guard each time, which is the correct simplification for studying selection in isolation but not the behaviour a real client should have.
- **Simplified weighting.** Tor’s full path-selection algorithm applies position-specific weight adjustments from the consensus (W_{gg} , W_{mg} , W_{ee} and related parameters) that rebalance relays across positions according to where capacity is scarce. This implementation weights by raw advertised bandwidth, capturing the central mechanism but not those corrections.
- **Exit policies are not checked.** Selection filters on the `Exit` flag but does not verify that the chosen exit’s policy actually permits the specific destination port being requested.
- **No measurement of the resulting distribution.** The natural next experiment is to build many circuits and plot the empirical relay selection frequency against advertised bandwidth, confirming the weighting behaves as intended and quantifying how concentrated selection actually is in practice.

5 Environment and tooling

The three assignments run on entirely different stacks, and reproducing any one of them depends on details that are easy to lose track of once the result is in hand. This section records what each actually needed.

5.1 Assignment 1

- **Guest OS:** FreeBSD 15.1-RELEASE on all three VMs (substituted for the assignment’s preferred 13.4, which is no longer distributed), plus the matching `src.txz` kernel source tree on VM2 for module builds.
- **Networking:** VirtualBox internal networks for the `em1/em2` routed path, a bridged adapter (`em0`) on each VM purely for host SSH access.
- **Firewall:** the base-system `pf` and `pfctl`, no third-party packages.

- **Kernel module build:** a two-line `bsd.kmod.mk` Makefile setting `KMOD` to `block_icmp` and `SRCS` to `block_icmp.c`, built with the base-system `make` against `/usr/src`.
- **HTTP service:** Apache (`apache24` package) serving the default document root on port 80.
- **Measurement:** `iperf3` for throughput, `curl` and `traceroute` for functional and path verification, `dmesg` for reading kernel-module console output.

5.2 Assignment 2

- **Platform:** Wireshark with Lua 5.3 or later support, the minimum the plugin API used here requires.
- **Deployment:** the plugin file is copied into Wireshark’s Personal Lua Plugins directory and loaded with `Ctrl+Shift+L` or a restart, after which it appears under *Help* → *About Wireshark* → *Plugins*.
- **Traffic sources:** Tor Browser, run against the live Tor network, for genuine handshakes; `mix_traffic.pcap` as the shared demonstration capture combining Tor and ordinary traffic.
- **No external libraries:** the plugin is pure Lua against Wireshark’s built-in `Field`, `Proto` and dissector APIs — there is no dependency to install beyond Wireshark itself.

5.3 Assignment 3

- **Runtime:** Python 3 against a locally running Tor client exposing `ControlPort 9051` and `SOCKSPort 9050`.
- **Libraries, pinned in `requirements.txt`:** `stem` version 1.8.2 for the control-protocol connection, `requests` version 2.32.3 with `PySocks` version 1.7.1 for SOCKS5-proxied HTTP, `beautifulsoup4` version 4.13.3 for scraping the Collector directory listing.
- **Inspection:** `nyx`, run alongside the scripts, to confirm live which relays a circuit actually uses.
- **Data source:** the Collector service’s recent-consensuses directory listing for Task 2’s raw consensus fetch, versus Task 1’s use of Tor’s own already-parsed internal view via `stem`, as discussed above.

6 Discussion

6.1 The common thread: building a thing is not demonstrating it

Working through the three assignments in sequence, the same problem appeared in each, in a different costume, and I did not anticipate it in any of them.

In Assignment 1 a firewall rule that blocks everything looks identical, from one side, to a correct rule — so I had to test both the traffic that should fail and the traffic that should still succeed; the same trap appeared again inside the assignment when the ICMP-permitting rule produced a firewall that was correct but not what I had intended. In Assignment 3, Tor helpfully attaches streams to circuits it chooses, so a demonstration that builds a custom circuit and then fetches a page produces convincing evidence for a claim that is false. In Assignment 2 the trap was

subtler: validating the plugin against a synthetic capture built from my own cipher list would have confirmed only that the list matches itself.

In all three cases the system’s normal, helpful behaviour was capable of manufacturing the result I was looking for. The work that made each result mean something was the work of removing that possibility — checking the reverse direction, disabling auto-attachment, testing against traffic I did not generate.

6.2 Layers determine what is possible, not just how it is done

The three assignments sit at three layers, and each layer decides what can be seen or controlled from it.

The kernel module sees every inbound packet before any filtering engine does, which is why it can be cheaper than `pf` — it carries no generality it does not need. The Wireshark plugin cannot see inside TLS at all, which is precisely why it must use the handshake, the only part of the exchange that cannot be encrypted because it is the negotiation of the encryption. The Tor work operates inside the protocol, where relay choice is visible as a decision rather than as traffic.

The pattern I take from this is that each layer’s blind spot forces the technique that works there. The plugin’s inability to decrypt is not a limitation it works around; it is the reason the fingerprint is the right signal.

6.3 Generality has a measurable price, and privacy has one too

Two of the assignments produced the same shape of result independently.

`pf` costs 5–6% throughput against a stateless hook doing nominally equivalent work, and that cost buys arbitrary rule matching, connection tracking and NAT. Tor’s bandwidth weighting buys usable performance and costs anonymity, because concentrating traffic on high-capacity relays means an adversary can increase their observation share by contributing bandwidth.

In both cases the trade-off is not a defect to be engineered away but the mechanism working as designed, and in both cases it is visible in the implementation — in `pf`’s state table, and in the width of an interval in a cumulative-weight draw.

6.4 Using a mechanism versus rebuilding it

A fourth thread runs across all three assignments, visible only once they are placed side by side: each one paired a use of an existing, trusted implementation with a from-scratch rebuild of the same underlying mechanism, and the rebuild is what produced the understanding.

Assignment 1 used `pf` in Task 1 and then built the equivalent filtering decision directly against `pfil(9)` in Task 2 — the layer `pf` is itself implemented on top of. Assignment 2 could have computed a full JA3 hash over five ClientHello fields, the general-purpose version of this technique; instead it isolates the one field, the ordered cipher-suite list, that carries Tor’s specific signal, which is only a defensible simplification because the general mechanism was understood well enough to know what could safely be dropped. Assignment 3 draws the sharpest version of the same contrast within a single deliverable: Task 1 takes Tor’s own already-computed view of the network via `stem` and uses it, while Task 2 discards that convenience and re-derives the same information by parsing the raw consensus document by hand.

In each case the “use it” half is not the weaker half — `pf` in production, a full JA3 hash in an intrusion-detection pipeline, and `stem`’s cached consensus view in an ordinary Tor client are

all the correct engineering choice for their actual purpose, which is getting a job done reliably rather than exposing how it works. But the assignments were not asking for that job; they were asking whether the mechanism underneath a trusted abstraction could be reconstructed and still produce the same behaviour, and that is a different, stronger claim than knowing how to call the abstraction correctly.

7 Conclusion

The three assignments taught me more together than they would have separately, because the same lesson arrived three times from different directions.

Technically, the durable results are these. Working at the `pfil(9)` layer clarified what `pf` actually is — not “the firewall” but one consumer of a general interception framework, whose generality is measurable as a 5–6% throughput cost. Fingerprinting the TLS handshake showed that encryption protects contents while the negotiation of that encryption identifies the software, so a user can encrypt everything and still be identified by how they set it up — and that the same signal is a countermeasure’s specification: change the handshake and the fingerprint is gone, which is exactly what pluggable transports do. And reimplementing Tor’s path selection turned bandwidth weighting from a fact I could state into a mechanism I can locate, along with the anonymity it spends, and made clear that a longer circuit is a demonstration of control rather than a stronger one.

Methodologically, the lesson is the one in §6.1, and it is the one I expect to keep. In each assignment the most convincing-looking evidence was available without establishing anything, because the system under test was willing to produce it for me. Default-deny, both-direction testing, disabling stream auto-attachment and validating against traffic I had not generated are all the same move: removing the path by which a demonstration could succeed for the wrong reason. The limitations recorded for each assignment are the other half of that discipline — marking, deliberately, where that removal was not yet done.